

Solar Tracking System

Please select your current program below:

- ☐ Mechanical Engineering
☐ Industrial Engineering
☒ ~~Mechatronics Engineering~~

Course Number	MTE604
Course Title	Electromechanical System Modeling and Design
Semester/Year	Winter 2026
Section Number	01
Group Number	

Instructor	Dr. Hossein Hassani
------------	---------------------

Project Report

Submission Date	2026-04-12
Due Date	2026-04-12

Student Name	Student ID (xxxx1234)	Signature*
Yasir Saeed	47624	YS
Mesbah Akife	49716	MA
Osman Asif	35477	OA

(Note: Remove the first 4 digits from your student ID)

**By signing above you attest that you have contributed to this submission and confirm that all work you have contributed to this submission is your own work. Any suspicion of copying or plagiarism in this work will result in an investigation of Academic Misconduct and may result in a "0" on the work, an "F" in the course, or possibly more severe penalties, as well as a Disciplinary Notice on your academic record under the Student Code of Academic Conduct, which can be found online at: <http://www.ryerson.ca/senate/policies/pol60.pdf>.*

Department of Mechanical, Industrial, and Mechatronics Engineering.....	1
Abstract:	2
Introduction:	2
Solar Tracking System:	3
System Overview.....	3
Mechanical Design.....	3
Mathematical Model.....	4
Controller Design and Gain Selection.....	5
Scenario Specific Modifications.....	6
Results:	7
Scenario 1: Baseline Tracking.....	7
Scenario 2: Wind Disturbance.....	9
Scenario 3: Sensor Error.....	10
Scenario 4: BONUS.....	13
Scenario 5: BONUS.....	14
Conclusion:	16
References:	17
Appendix:	18

Abstract:

This project focused on building and testing a solar tracking system in Gazebo that can rotate a solar panel and follow the sun's path over a 25 second simulation. The main goal was to create a realistic mechanical model with proper physics, link properties, joint limits, as well as damping and actuation, then design a controller that could track the reference motion with small error. To do this, the system was modeled as a second order rotational mass damping system, converted into a state space form, and used to obtain PD control gains for a critically damped response. The tracker was then tested in the normal case, under wind disturbance, and under sensor inaccuracy to study how the performance changed in different conditions. The results were evaluated using tracking graphs and error measures such as MAE and RMSE.

Overall, the project showed us that the controller could follow the desired trajectory well in the normal case, all while the wind and sensor imperfections increased the tracking error as expected, which helped show the limits and practical behavior of the system more clearly.

Introduction:

In the real world, solar energy systems work best when the panel is pointed toward the sun as closely as possible. A fixed panel can still collect power, but it cannot keep the best angle during the whole day, and that's why these systems are so important for energy collection. That is why solar tracking systems are important. They help the panel rotate with the sun's path so more energy can be captured. In real systems, this is usually done with a mechanical support, a motor or actuator, sensing, and a controller that keeps adjusting the panel angle. This project focuses on that same idea by simulating a solar tracker that automatically rotates and follows the sun across the sky.

The background of this project is based on building a realistic solar tracking model in Gazebo. The tracker is not just a simple moving part. It includes a simulation world, physical links, joint constraints, and plugins that allow actuation and sensing. In this project the solar panel is mounted on a base and connected through a revolute joint so it can rotate about one axis. The model also includes many physical properties like mass, inertia, as well as damping and friction, joint limits, and actuator effort, which are all important because they affect how the panel moves in a more realistic way. This is also important in the sense of this project's requirements, since the project is expected to have things like a proper environment setup, correct panel properties, correct plugin integration, and stable joint behavior.

To address the problem, this project uses Gazebo and SDF to build the mechanical system, then uses a Python script as the high level controller. The script generates the reference path for the sun over a 25 second simulation and sends commands so the panel can track that motion. To make the controller meaningful, the rotational system is modeled as a second order mass damper system, then converted into state space form so critically damped PD gains can be found. After that, the tracker is tested in different conditions including the normal case, wind disturbance, and sensor inaccuracy. These scenarios help show not only whether the panel can track the reference, but also how well it handles more realistic problems that may happen in practice.

Solar Tracking System:

System Overview

This project represents the design and simulation of a single-axis solar tracking system implemented in Gazebo Fortress. The purpose of this system is to rotate a solar panel such that it follows the simulated motion of the sun, representing how a real solar tracking system maintains alignment with the sun for improved energy capture. The project consists of a mechanical model in Gazebo and a controller written in python that commands the solar panels motion and records the tracking performance and errors.

The python script generates a continuous mathematical trajectory that represents the sun's path across the sky over a 25-second simulation period using the following reference function,

$$\theta_{ref}(t) = -\cos\left(t \cdot \frac{\pi}{25}\right)$$

To evaluate performance, the actual panel angle was compared against the reference angle and the tracking error was later quantified using MAE and RMSE.

Mechanical Design

The mechanical structure of the solar tracker was designed to simulate a robust 1DOF industrial array. The system was defined within the tracker_world environment and utilized the Simulation Description Format (.SDF) to ensure accurate dynamic behavior. Furthermore, the simulation environment was configured with appropriate step times of 0.001s and a real-time factor of 1.0, providing the temporal resolution necessary for control. In addition to the simulation plug-ins for physics, the world required plug-ins for scene broadcasting, user commands as well as a light source to model the sun & base plate to support the tracker.

The physical solar tracker consists of two primary links, a base link which is a stationary vertical pole, with $r=0.1\text{m}$, $l=1.0\text{m}$. This base acts as an anchor to the world by a fixed point. The second link is the actual solar panel, a rectangular array with dimensions of $2.0 \times 1.0 \times 0.05\text{m}$, with a mass of 10.0kg . Specific inertial properties were used to determine the solar panel's strength to resist rotational acceleration during tracking. Furthermore, for rotation, a revolute joint was used called tracking_joint and was connected to the fixed joint. This revolute joint constrained the motion to a single degree of freedom about the Y axis and limits of $\pm 90^\circ$ to prevent unrealistic rotational movement. The joint was given a maximum effort of $100 \text{ N} \cdot \text{m}$, and a maximum angular velocity of 1.0 rad/s . Furthermore, to ensure real mechanical behaviour, friction and damping were included with values of $0.1 \text{ N} \cdot \text{m}$, and $0.5 \text{ N} \cdot \text{M} \cdot \text{s/rad}$, respectively. The mechanical design and parameters specified help create a realistic single-axis tracker suitable for control analysis and validation.

Mathematical Model

The model was constrained to rotate only about the y-axis through the revolute joint. The dynamics of the rotation were represented by the 2nd-order mass damper system,

$$I\ddot{\theta} + b\dot{\theta} = \tau$$

Where I is the moment of inertia of the panel about the y-axis, b is the viscous damping coefficient, and τ is the actuator torque at the revolute joint.

From the mechanical model, the simplified plant model used for the controller was,

$$2.0(\ddot{\theta}) + 0.5(\dot{\theta}) = \tau$$

To represent the system in state-space form, the states were chosen as,

$$x_1 = \theta, x_2 = \dot{\theta}$$

The resulting state equations are,

$$\begin{aligned} \dot{x}_1 &= x_2 \\ \dot{x}_2 &= -\frac{b}{I}x_2 + \frac{1}{I}\tau \end{aligned}$$

In matrix form this is,

$$\dot{x} = \begin{bmatrix} 0 & 1 \\ 0 & -\frac{b}{I} \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} + \begin{bmatrix} 0 \\ \frac{1}{I} \end{bmatrix} \tau$$

The state space model was used to design a controller that forces the panel angle to follow the reference function of the sun's trajectory

Controller Design and Gain Selection

The projects objective is to ensure the solar panel tracks the reference sun trajectory,

$\theta_{ref}(t) = -\cos(t \cdot \frac{\pi}{25})$, with minimal error and high stability. The state-feedback controller was designed for a critically damped response for which $\zeta = 1.0$, this creates a PD controller. Control law is defined as

$$\tau = K_p(\theta_{ref} - \theta) + K_d(\dot{\theta}_{ref} - \dot{\theta})$$

Substituting into the original plant model gives us:

$$I\ddot{\theta} + (b + K_d)\dot{\theta} + K_p\theta = K_d\dot{\theta}_{ref} + K_p\theta_{ref}$$

$$Is^2 + (b + K_d)s + K_p = 0$$

$$s^2 + \frac{(b+K_d)}{I}s + \frac{K_p}{I} = 0$$

After getting the equation to match the general second order form: $s^2 + 2\zeta w_n s + w_n^2 = 0$,

We equate the components:

$$w_n^2 = \frac{K_p}{I}$$

$$2\zeta w_n = \frac{(b+K_d)}{I}$$

Solve for each Gain:

$$\frac{w_n^2}{I} = K_p$$

$$2\zeta w_n I - b = K_d$$

Plugging in the parameters specified from the lab manual as well as the SDF file, we obtain the proportional and derivative gains:

$$K_p = 18.0$$

$$K_d = 11.5$$

These values were implemented in the plugin to maintain precise alignment with the sun's trajectory.

Scenario Specific Modifications

After the baseline SDF file and controller were made for scenario 1. The same mechanical model and PD controller were reused across the other scenarios, with additional scenario specific changes added in the python controller script.

In scenario 2, a wind disturbance was added to the revolute tracking joint to test the robustness of the controller against external loading. The disturbance torque was generated with 3 components, a continuous sinusoidal term to represent rhythmic wind, a random term to represent turbulent fluctuations, and occasional large spikes to represent sudden gusts. The combined torque was then applied to the revolute tracking joint during the simulation.

In scenario 3, the effect of sensor inaccuracy was added. Rather than using a perfect joint-angle reading for evaluation, the measured angle was intentionally corrupted in the controller code using 3 imperfections: Gaussian noise, quantization, and slow drift. Gaussian noise simulated sensor variation, quantization reproduced the limited resolution of a digital encoder by rounding the signal to discrete steps, and the slow drift term represented a gradual time-varying bias in the measurement.

Results:

Scenario 1: Baseline Tracking

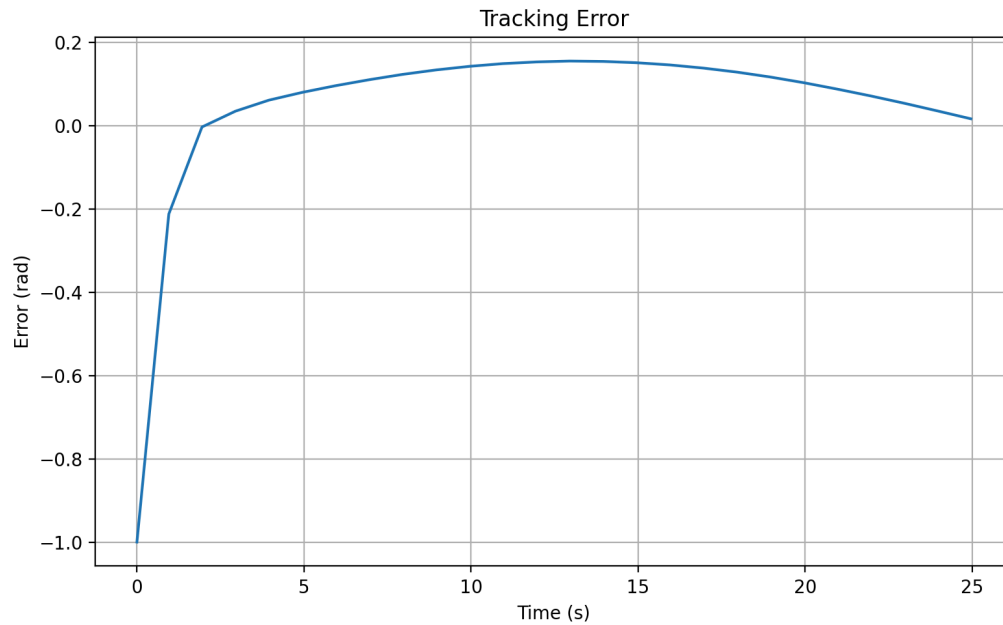


Figure 1. Tracking Error Scenario 1

For the Scenario 1 plot above, the tracking error starts with a large negative value of about -1.0 rad at the beginning and then it quickly rises to around 0 by what looks to be 2 seconds. After that, the error stays relatively small and smooth, becomes slightly positive, reaches its highest value at about 0.15 rad around the middle of the simulation and then slowly drops again toward the end. From the recorded data in the appendix the baseline case gives an MAE of about 0.141 rad and an RMSE of about 0.228 rad which shows that the controller was able to follow the sun path reasonably well in this no disturbance case.

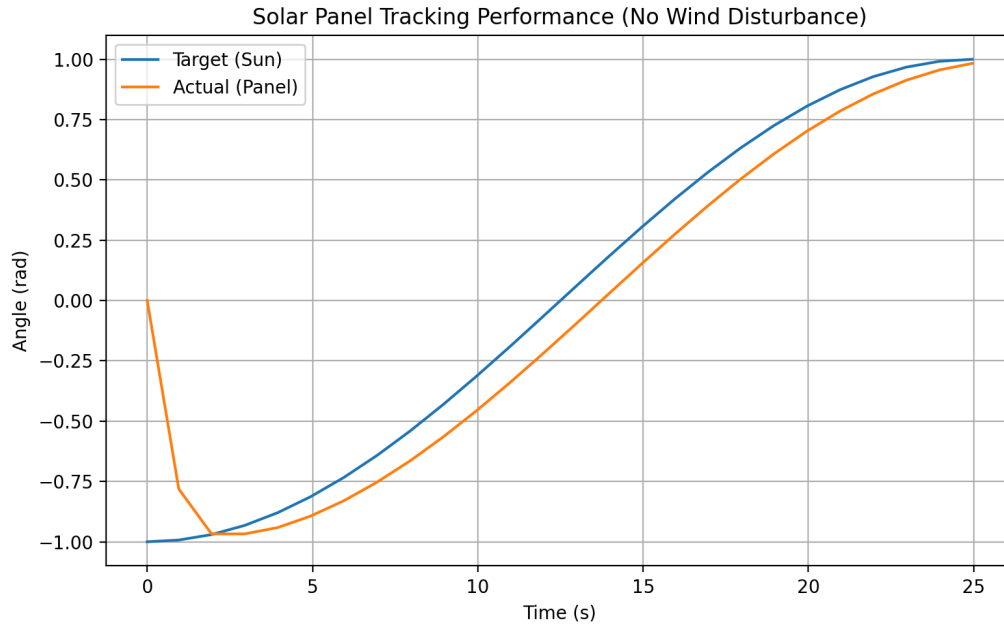


Figure 2. Solar Panel Tracking Performance No Wind Disturbance Scenario 1

From the tracking performance plot above, the actual panel angle follows the reference curve closely for most of the simulation but there is a clear lag especially at the start and through the middle part of the motion. The biggest reason for the initial error is that the target begins near -1 rad while the actual panel starts near 0 rad so the controller first has to catch up from that mismatch. After that, the response becomes smoother and more stable which makes sense because this case has no wind disturbance and no sensor corruption. The remaining error is likely due to the physical inertia of the panel and the damping in the joint which prevent the system from matching the changing reference instantly.

Overall, this Scenario 1 shows that the controller works properly in the baseline case and gives a good reference point for comparing the disturbed scenarios later.

Scenario 2: Wind Disturbance

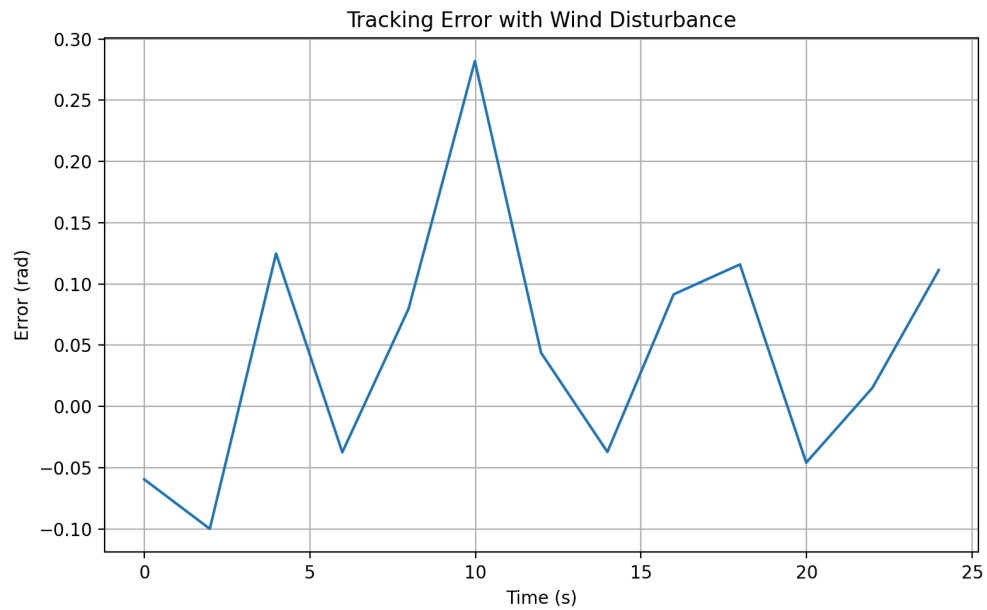


Figure 3. Tracking Error with Wind Disturbance Scenario 2

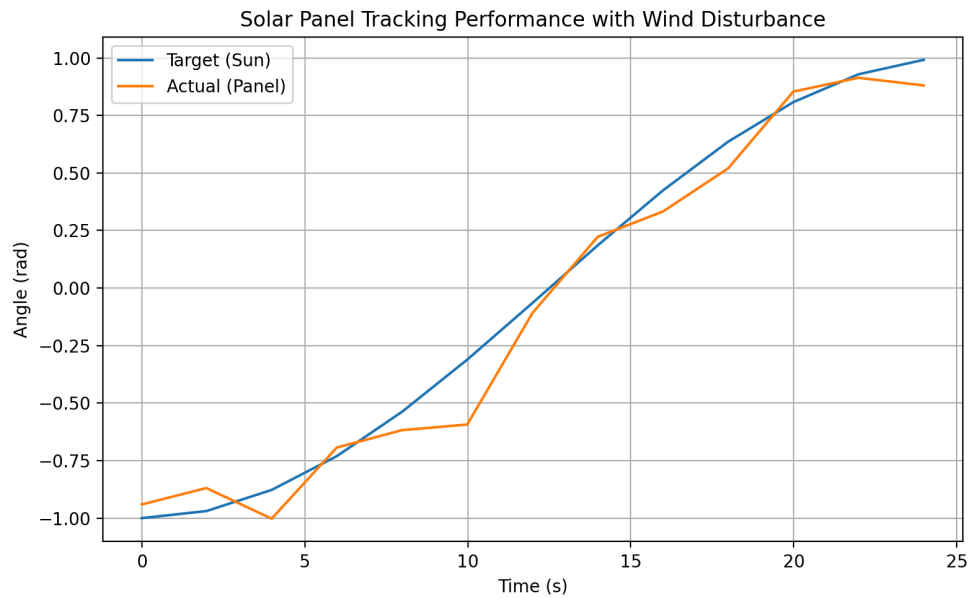


Figure 4. Solar Panel Tracking Performance with Wind Disturbance Scenario 2

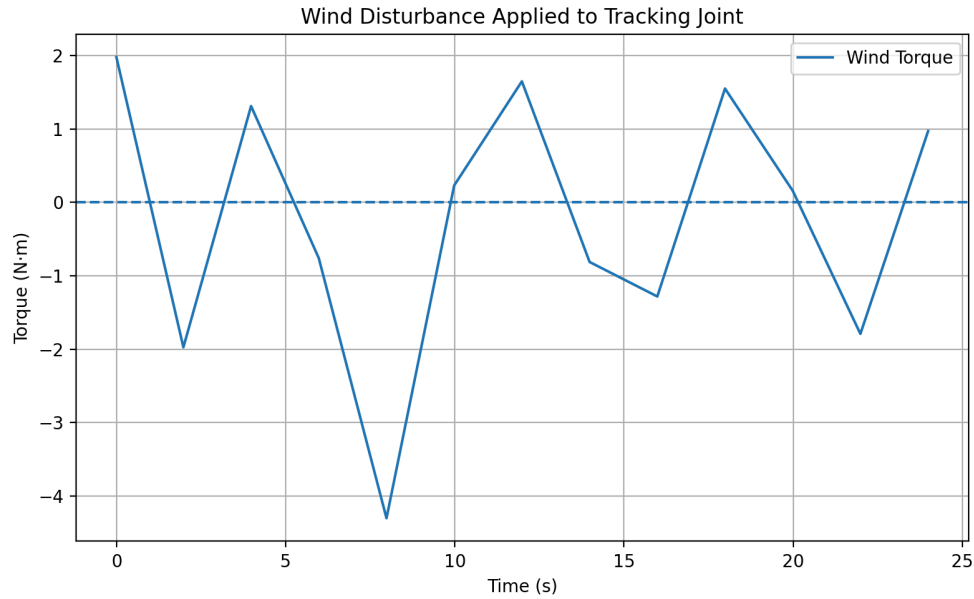


Figure 5. Wind Disturbance Applied to Tracking Joint Scenario 2

For the Scenario 2 plot above, the tracking error is no longer smooth like the baseline case. It oscillates between small negative and positive values for most of the run with the largest peak reaching about 0.28 rad around 10 seconds. Most of the error stays within roughly -0.10 to 0.12 rad, but there are a few sharper changes caused by the disturbance acting on the joint.

From the tracking plot, the panel still follows the general sun path but it does it with more waviness and small deviations than Scenario 1. This makes sense because Scenario 2 adds wind torque made from oscillation, turbulence, and gust like spikes, so the controller is always correcting against an extra external load. Even with that, the system still remains stable and tracks the overall reference quite well which shows the controller is reasonably robust even though the disturbance clearly makes the motion less clean and increases the moment to moment error.

Scenario 3: Sensor Error

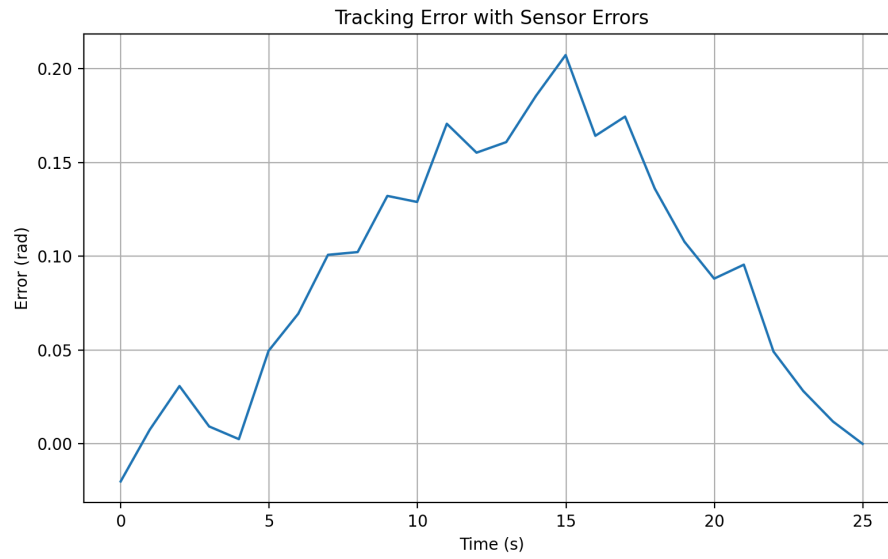


Figure 6. Tracking Error with Sensor Errors Scenario 3

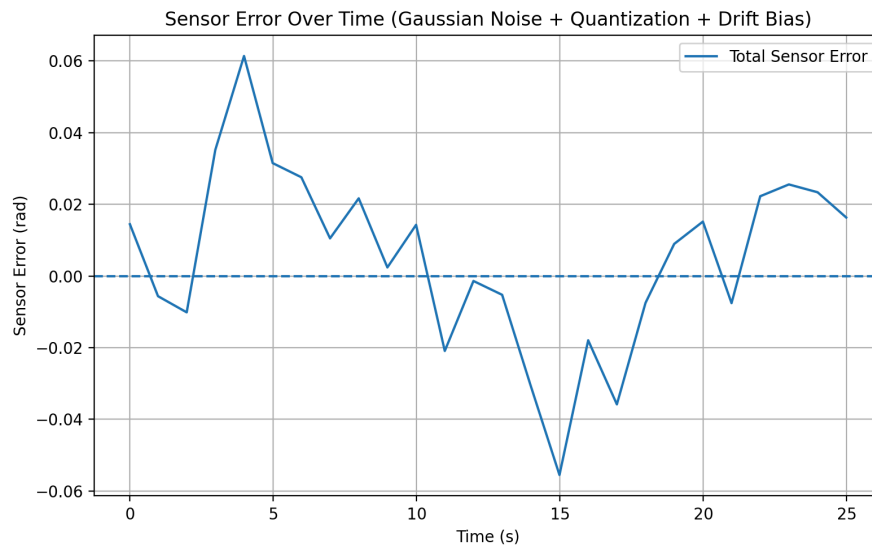


Figure 7. Sensor Error Over Time Scenario 3

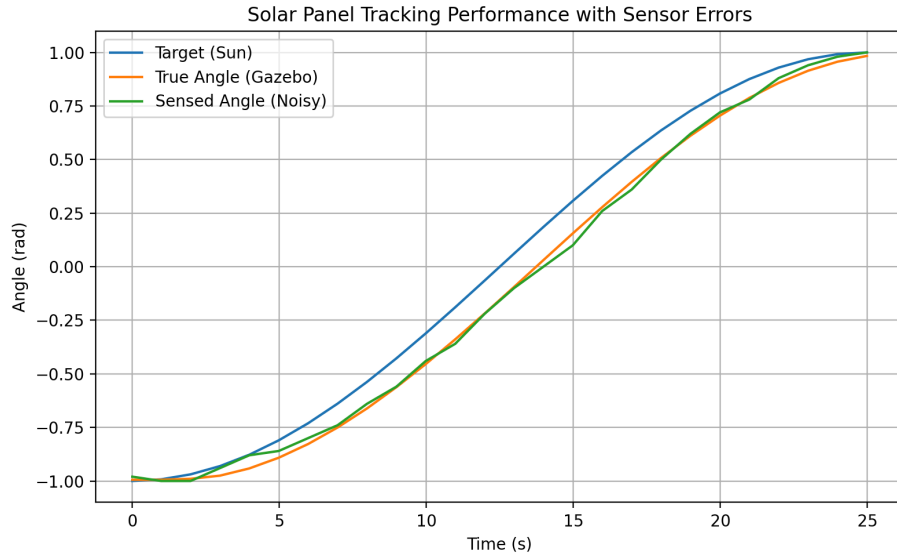


Figure 8. Solar Panel Tracking Performance with Sensor Errors Scenario 3

For the Scenario 3 plot above, the tracking error stays fairly small for most of the simulation but it is more biased than in the baseline case. It starts slightly below zero and then gradually rises and reaches a peak of about 0.21 rad around the middle of the run, before dropping back down toward zero near the end. The sensor error plot also shows that the measured angle was being disturbed in both directions with the total sensor error staying roughly within about ± 0.06 rad over time.

From the tracking performance plot, the true panel angle still follows the target curve quite well and the sensed angle stays close to it even though it is noisy. This makes sense because Scenario 3 adds Gaussian noise and quantization, as well as slow drift to the measured joint position so the controller is working with imperfect feedback instead of a perfect sensor reading. Even with that, the system does not become unstable and the tracking remains reasonably accurate. This means the controller is fairly robust to the sensor inaccuracy. The main effect is not a complete loss of tracking but a slight increase in error and a slightly less clean response.

Scenario 4: BONUS

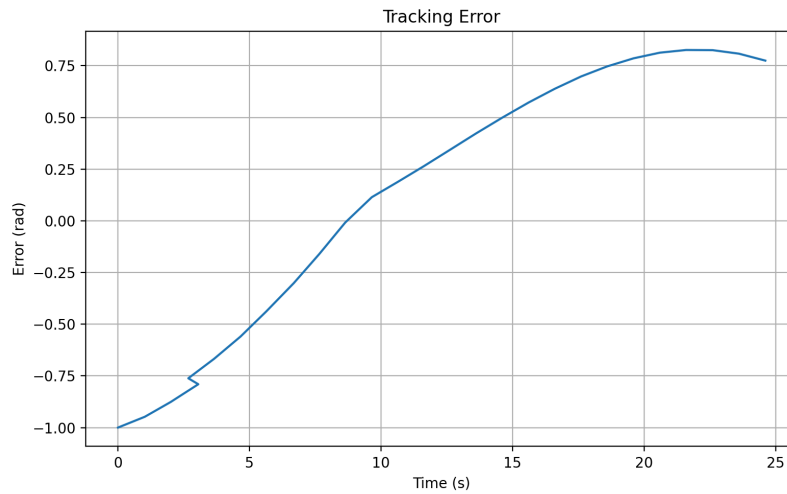


Figure 9. Tracking Error Scenario 4

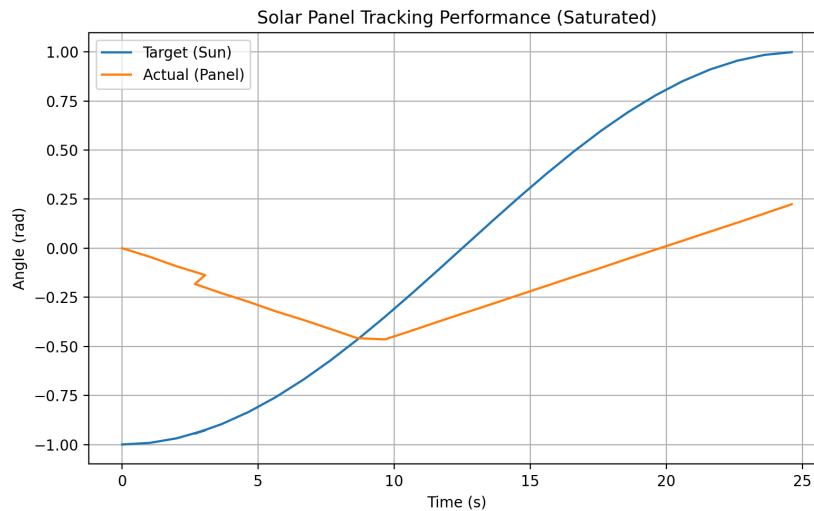


Figure 10. Solar Panel Tracking Performance (Saturated) Scenario 4

For Scenario 4, the tracking error becomes much larger and more sustained than in the earlier cases. It starts near -1.0 rad and crosses into positive error later in the run, and then stays quite high for a large part of the simulation which shows that the panel is not able to keep up with the target path once the actuator limits begin to matter. From the saturated tracking plot the actual panel angle moves much more slowly than the reference and never fully catches it. This makes sense because Scenario 4 limits the maximum motor torque and also restricts the joint speed so even if the controller demands a stronger correction the actuator cannot deliver it fast enough.

Scenario 5: BONUS

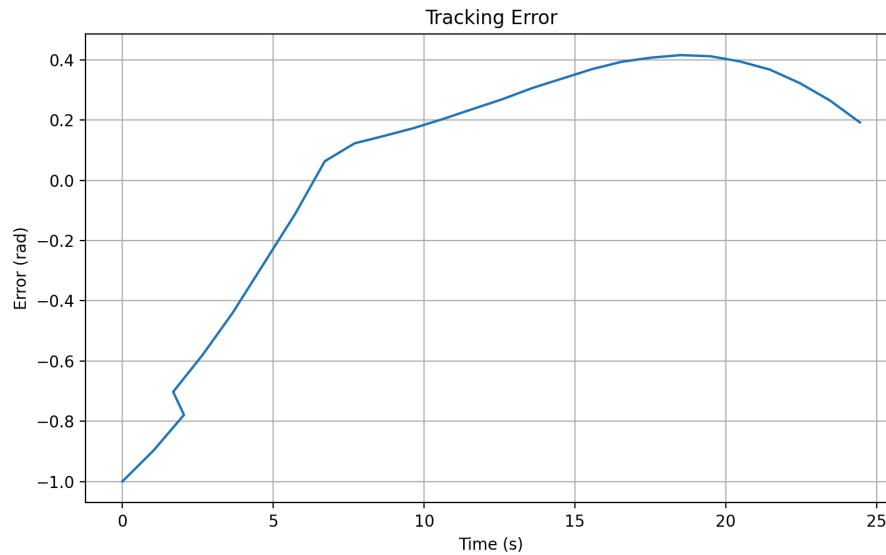


Figure 11. Tracking Error Scenario 5

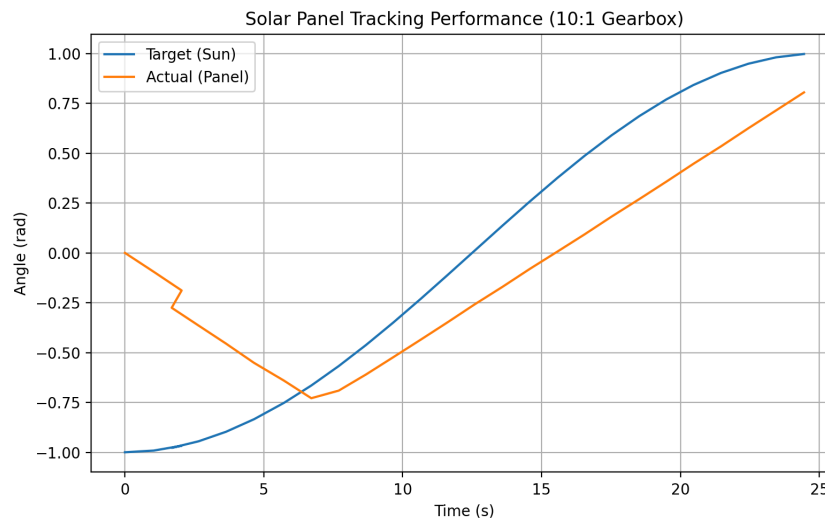


Figure 12. Solar Panel Tracking Performance (10:1 Gearbox) Scenario 5

For Scenario 5, the tracking error is still noticeable but it is clearly better than the saturated case. The error begins large and negative then rises gradually, becomes positive, and stays moderate through the rest of the run instead of becoming as severe as Scenario 4. From the 10:1 gearbox tracking plot, the panel still lags behind the sun path but it follows the overall shape better and ends much closer to the target. This is reasonable because the gearbox increases the effective torque available to the system so the panel can respond better than a fully saturated actuator, but the weak motor setup still makes the response slower than the baseline case.

Conclusion:

This project gave a good look into how a solar tracking system can be modeled and controlled in simulation. The main goal was to build the tracker in Gazebo, define the mechanical and physical properties correctly, and then design a controller that could make the panel follow the sun's path. By doing this project, it became clearer how the mechanical model and the control model depend on each other. Even if the controller design is good, the system would still need the proper mass, inertia, damping, joint limits, and plugin setup or else the simulation will not behave the way it should. Everything must be in place for system correctness. This project also showed how a second order rotational model can be used in a practical way to obtain controller gains and improve the system response.

One important thing learned from this project was that control design is not just about getting equations and gains. Rather it also involves understanding the physical system, like how the simulation environment works, and how disturbances or sensor issues can affect tracking. Building the model and testing it through different scenarios helped us connect the theory to something more realistic, which made the whole process more useful and memorable for when we need it in the real world. It also helped show why tracking systems in real renewable energy applications need both a solid mechanical design and a stable controller.

At the end, the results showed that the controller worked well when the system had normal operating conditions and it also remained fairly stable when wind disturbance and sensor inaccuracy were added. The weakest performance happened when actuator limits slowed the system down too much while the gearbox case showed that torque improvement helped but it still did not match the baseline tracking quality.

If we were to do this project again, something that could be improved is spending more time tuning and testing the system earlier in the process. That would make it easier to catch issues with the model, the plugins, or the controller before moving into the more difficult scenarios, which is what ended up happening in our case. The troubleshooting takes a while in terms of going back and trying to find the issue, or how to make the system better. Another improvement would be to spend more time refining and testing the optional scenarios, especially actuator saturation and gearbox behavior, so their implementation could be made even stronger and closer to real solar tracking behavior. In general, the project was a good way to understand modeling, simulation, and control together in one full system.

References

- [1] H. Hassani, “Project Description,” Toronto Metropolitan University,
<https://courses.torontomu.ca/d2l/le/content/1133195/viewContent/6925081/View>
(accessed Apr. 12, 2026).
- [2] H. Hassani, “project rubric,” Toronto Metropolitan University,
<https://courses.torontomu.ca/d2l/le/content/1133195/viewContent/6959367/View>
(accessed Apr. 12, 2026).

Appendix

time_s	theta_ref_rad	theta_actual_rad	error_rad
4.77E-07	-1	-4.99E-05	-0.99995
0.954021	-0.99282	-0.78098	-0.21184
1.946612	-0.97023	-0.96744	-0.00278
2.946993	-0.93221	-0.96739	0.035187
3.947386	-0.87947	-0.94114	0.061668
4.946937	-0.81292	-0.89331	0.080393
5.947547	-0.73346	-0.82995	0.096485
6.94763	-0.64248	-0.75326	0.110778
7.948591	-0.54127	-0.66486	0.123592
8.949106	-0.43156	-0.56585	0.134293
9.94905	-0.3151	-0.45803	0.142926
10.94906	-0.19367	-0.34312	0.149457
11.94962	-0.06911	-0.22265	0.153539
12.95221	0.056796	-0.09886	0.155656
13.95164	0.181409	0.026664	0.154745
14.95097	0.303151	0.151501	0.15165
15.95131	0.420235	0.273889	0.146346
16.95172	0.530694	0.391954	0.138741
17.95303	0.632865	0.503764	0.129102
18.95367	0.724971	0.60765	0.117321
19.95376	0.805588	0.701924	0.103664
20.95305	0.873449	0.78513	0.088319

21.9543	0.927647	0.855718	0.071929
22.95474	0.967153	0.912747	0.054406
23.95477	0.991386	0.955498	0.035889
24.95544	0.999984	0.983035	0.016949

Table 1. Tracking Data Scenario 1

time_s	theta_ref_rad	theta_actual_rad	error_rad	wind_torque
1.19E-06	-1	-0.94043	-0.05957	1.978884
1.979838	-0.96921	-0.86926	-0.09995	-1.97394
3.980377	-0.87749	-1.00233	0.124842	1.312384
5.980935	-0.73061	-0.69316	-0.03745	-0.75952
7.982038	-0.53773	-0.61759	0.079863	-4.30228
9.982363	-0.31112	-0.59311	0.281991	0.230855
11.98282	-0.06494	-0.10866	0.043718	1.649038
13.98447	0.185464	0.222614	-0.03715	-0.81168
15.98497	0.424069	0.332533	0.091536	-1.28157
17.98576	0.636044	0.520103	0.115941	1.550777
19.98684	0.808044	0.853927	-0.04588	0.158711
21.98656	0.929153	0.913918	0.015235	-1.79077
23.98781	0.991922	0.880587	0.111334	0.971833

Table 2. Tracking Data Scenario 2

time_s	theta_ref_rad	theta_actual_rad	theta_measured_rad	tracking_error_rad	sensor_error_rad
9.54E-07	-1	-0.99444	-0.98	-0.02	0.014443

0.980273	-0.99242	-0.99436	-1	0.007578	-0.00564
1.980852	-0.96918	-0.98985	-1	0.030821	-0.01015
2.979852	-0.93071	-0.97522	-0.94	0.009294	0.035221
3.98078	-0.87747	-0.9414	-0.88	0.002532	0.061404
4.980804	-0.81043	-0.89151	-0.86	0.049567	0.031509
5.980649	-0.73063	-0.82756	-0.8	0.069369	0.027555
6.981063	-0.63926	-0.7505	-0.74	0.100744	0.010501
7.98148	-0.53779	-0.66167	-0.64	0.10221	0.021672
8.981701	-0.42786	-0.56239	-0.56	0.132141	0.002388
9.982715	-0.31108	-0.45427	-0.44	0.128918	0.014273
10.98355	-0.18941	-0.33907	-0.36	0.170589	-0.02093
11.9838	-0.06482	-0.2186	-0.22	0.155177	-0.0014
12.98413	0.060801	-0.09474	-0.1	0.160801	-0.00526
13.98392	0.185396	0.030718	0	0.185396	-0.03072
14.98454	0.307169	0.155552	0.1	0.207169	-0.05555
15.98575	0.424158	0.277957	0.26	0.164158	-0.01796
16.98653	0.534397	0.395838	0.36	0.174397	-0.03584
17.98608	0.636075	0.507506	0.5	0.136075	-0.00751
18.98562	0.72773	0.611056	0.62	0.10773	0.008944
19.98701	0.808056	0.704822	0.72	0.088056	0.015178
20.9866	0.875494	0.787584	0.78	0.095494	-0.00758
21.98681	0.929165	0.857746	0.88	0.049165	0.022254
22.98709	0.968178	0.914447	0.94	0.028178	0.025553
23.98799	0.991924	0.956631	0.98	0.011924	0.023369
24.98786	0.999999	0.983684	1	-1.16E-06	0.016316

Table 3. Tracking Data Scenario 3

time_s	theta_ref_rad	theta_actual_rad	error_rad
4.77E-07	-1	-9.50E-07	-1
1.018471	-0.99182	-0.04395	-0.94787
2.006789	-0.96837	-0.09235	-0.87602
3.052157	-0.92734	-0.1369	-0.79044
2.672875	-0.94412	-0.18245	-0.76167
3.649523	-0.89667	-0.2288	-0.66787
4.658491	-0.83349	-0.27375	-0.55974
5.639074	-0.75926	-0.3213	-0.43796
6.680123	-0.66787	-0.3662	-0.30168
7.649881	-0.57244	-0.41149	-0.16095
8.638381	-0.46644	-0.45804	-0.0084
9.64927	-0.35062	-0.46487	0.114246
10.62864	-0.233	-0.42058	0.18758
11.61941	-0.11043	-0.37403	0.263598
12.62348	0.015516	-0.32808	0.343597
13.62012	0.140294	-0.28293	0.423225
14.61189	0.262283	-0.23688	0.499164
15.61091	0.381048	-0.19113	0.572178
16.60561	0.49334	-0.14523	0.638571
17.60545	0.598454	-0.09968	0.698135
18.59447	0.693153	-0.05348	0.746633
19.59648	0.778185	-0.00753	0.785715

20.59816	0.850873	0.03777	0.813103
21.59817	0.910011	0.08392	0.826091
22.60895	0.955198	0.12997	0.825229
23.60413	0.984655	0.17637	0.808285
24.60832	0.998789	0.22407	0.774719

Table 4. Tracking Data Scenario 4

time_s	theta_ref_rad	theta_actual_rad	error_rad
4.77E-07	-1	-4.95E-06	-1
1.040111	-0.99147	-0.09513	-0.89634
2.041043	-0.96729	-0.18843	-0.77886
1.682578	-0.97773	-0.27523	-0.7025
2.653594	-0.94492	-0.36472	-0.58019
3.639408	-0.89723	-0.45511	-0.44211
4.646272	-0.83434	-0.55151	-0.28283
5.722811	-0.75237	-0.64051	-0.11186
6.709078	-0.66516	-0.7289	0.06374
7.69633	-0.56765	-0.69079	0.123146
8.685769	-0.46117	-0.60918	0.148017
9.664853	-0.34879	-0.52276	0.173977
10.64047	-0.23155	-0.43596	0.204409
11.61163	-0.1114	-0.34846	0.237059
12.58615	0.010826	-0.25856	0.269389
13.58822	0.136324	-0.17056	0.306887
14.5735	0.257626	-0.08046	0.338089

15.57109	0.376416	0.006737	0.369679
16.54971	0.487218	0.093137	0.394081
17.5241	0.590233	0.182437	0.407796
18.51557	0.685972	0.269737	0.416235
19.49247	0.76991	0.357537	0.412373
20.46354	0.841864	0.446637	0.395227
21.46048	0.902701	0.534637	0.368064
22.44505	0.9489	0.625337	0.323563
23.44483	0.980965	0.715037	0.265928
24.44579	0.997576	0.805037	0.192539

Table 5. Tracking Data Scenario 5